

**BAHAN PRESENTASI KELOMPOK
LAPORAN HASIL HASH TABLE**



Disusun Oleh :

I Komang Dika Gus Septa (42530015)

I Wayan Yoga Karang Saputra (42530013)

Dosen :

Ni Luh Putu Ika Candrawengi S.Stat., M.Stat

**PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK DAN INFORMATIKA
UNIVERSITAS PENDIDIKAN NASIONAL
2026**

DAFTAR ISI

BAHAN PRESENTASI KELOMPOK LAPORAN HASIL HASH TABLE ...	i
DAFTAR ISI	i
DAFTAR GAMBAR	iii
1.1 Latar Belakang	1
1.2 Rumusan Masalah	2
1.3 Tujuan Penulisan	2
BAB II PEMBAHASAN	3
2.1 Pengertian <i>Hash Table</i>	3
2.2 Pengertian <i>Hash Function</i>	4
2.3 Pengertian <i>Hash Value</i>	5
2.4 Pengertian <i>Linear Probing</i>	5
2.5 Pengertian <i>Quadratic Probing</i>	6
2.6 Pengertian <i>Double Hashing</i>	7
2.7 Pengertian <i>Linked List</i>	8
2.8 Penjelasan Mentahan Program	8
2.8.1 Metode <i>Linked List Chaining</i>	8
2.8.1.1 Konsep Dasar Linked List Chaining :	8
2.8.1.2 Rumus yang Digunakan	9
2.8.1.3 Langkah-langkah Mencari Data	10
2.8.2 Metode <i>Quadratic</i>	11
2.8.2.1 Konsep Dasar Quadratic	11
2.8.2.2 Rumus yang Digunakan	11
2.8.2.3 Langkah-langkah Mencari Data	12
2.8.3 Kesimpulan dari Metode <i>Quadratic & Linked List Chaining</i>	13
2.9 Penjelasan <i>Program Coding</i>	14
2.10 Hasil <i>Output</i>	18

BAB III PENUTUP	19
3.1 Kesimpulan	19
3.2 Saran	20
DAFTAR PUSTAKA.....	21

DAFTAR GAMBAR

Gambar 2. 1 <i>Hash Table</i>	3
Gambar 2. 2 <i>Hash Function</i>	4
Gambar 2. 3 <i>Hash Value</i>	5
Gambar 2. 4 <i>Linear Probing</i>	5
Gambar 2. 5 <i>Quadratic Probing</i>	6
Gambar 2. 6 <i>Double Hashing</i>	7
Gambar 2. 7 <i>Linked List</i>	8
Gambar 2. 8 <i>Metode Linked List</i>	9
Gambar 2. 9 <i>Output</i>	18

BAB I

PENDAHULUAN

1.1 Latar Belakang

Coding adalah proses menulis serangkaian perintah atau instruksi ke dalam bahasa pemrograman agar komputer dapat melakukan tugas tertentu. Instruksi tersebut ditulis dalam bentuk kode, yang memiliki aturan atau sintaks khusus yang membuat komputer dapat memahami dan menjalankannya. *Coding* memungkinkan seseorang untuk mengontrol cara sebuah program beroperasi, mulai dari menerima *input*, memproses data, dan menghasilkan *output*. Berbagai bidang teknologi menggunakan *coding*, termasuk pembuatan aplikasi, *website*, *game*, pengolahan data, sistem informasi, dan kecerdasan buatan. Seorang programmer tidak hanya menulis kode, tetapi juga berpikir logis dan sistematis untuk menyelesaikan masalah selama proses *coding*. *Coding* adalah keterampilan penting di era teknologi saat ini karena membantu mengubah ide atau kebutuhan menjadi solusi digital yang dapat digunakan oleh komputer.

C++ adalah bahasa pemrograman yang digunakan untuk mengembangkan berbagai jenis aplikasi komputer, mulai dari program sederhana hingga sistem yang kompleks. Bahasa ini merupakan pengembangan dari bahasa C yang dibuat oleh Bjarne Stroustrup pada tahun 1980-an dengan menambahkan fitur modern seperti *Object-Oriented Programming (OOP)*. C++ termasuk bahasa multi-paradigma karena mendukung beberapa pendekatan pemrograman, seperti prosedural, berorientasi objek, dan generik. Selain itu, C++ dikenal memiliki performa yang cepat dan efisien karena dapat berinteraksi langsung dengan perangkat keras serta memberikan kendali penuh terhadap pengelolaan memori melalui

pointer dan alokasi dinamis. Bahasa ini juga bersifat portabel, sehingga dapat digunakan di berbagai sistem operasi seperti *Windows* dan *Linux*.

1.2 Rumusan Masalah

1. Apa itu *Hash Table*?
2. Apa itu *Hash Function*?
3. Apa itu *Hash Value*?
4. Apa itu *Linear Probing*?
5. Apa itu *Quadratic Probing*?
6. Apa itu *Double Hashing*?
7. Apa itu *Linked List*?
8. Bagaimana penjelasan mengenai mentahan programnya?
9. Bagaimana penjelasan mengenai *program* codingnya?
10. Bagaimana hasil outputnya?

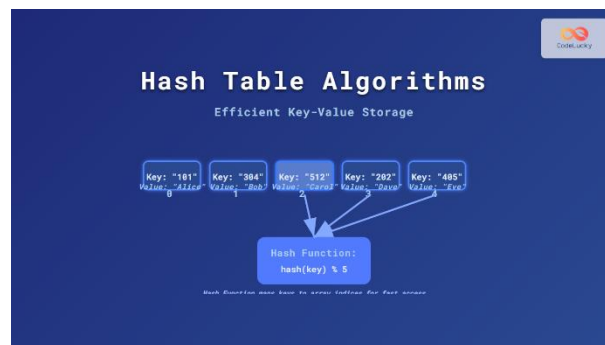
1.3 Tujuan Penulisan

1. Menjelaskan pengertian *Hash Table*.
2. Menjelaskan pengertian *Hash Function*.
3. Menjelaskan pengertian *Hash Value*.
4. Menjelaskan pengertian *Linear Probing*.
5. Menjelaskan pengertian *Quadratic Probing*.
6. Menjelaskan pengertian *Double Hashing*.
7. Menjelaskan pengertian *Linked list*.
8. Menjelaskan mengenai mentahan program.
9. Menjelaskan mengenai program codingnya.
10. Memaparkan hasil *output*.

BAB II

PEMBAHASAN

2.1 Pengertian *Hash Table*



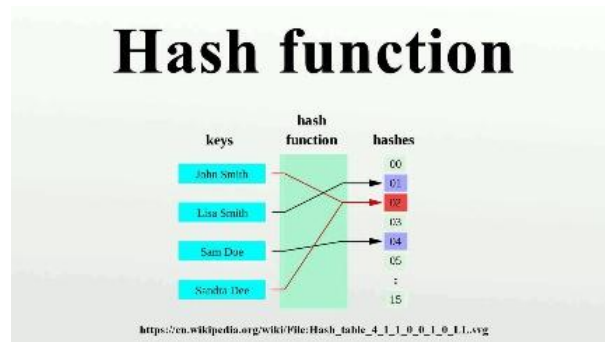
Gambar 2. 1 Hash Table

Hash table adalah struktur data yang digunakan untuk menyimpan data dalam bentuk pasangan *key* (kunci) dan *value* (nilai), di mana setiap data disimpan pada posisi indeks tertentu yang ditentukan oleh fungsi *hash*. Fungsi *hash* ini mengubah kunci menjadi angka indeks di dalam *array*, sehingga proses penyimpanan dan pencarian data dapat dilakukan dengan sangat cepat. Dengan cara kerja tersebut, hash table mampu memberikan waktu akses yang hampir konstan atau $O(1)$, sehingga sangat efektif digunakan untuk mengelola data dalam jumlah besar.

Namun, dalam penggunaannya, *hash table* dapat mengalami *collision*, yaitu ketika dua kunci yang berbeda menghasilkan indeks yang sama. Untuk mengatasi hal ini, digunakan beberapa metode seperti *chaining*, yaitu menyimpan beberapa data dalam satu indeks menggunakan struktur seperti *linked list*, atau *probing*, yaitu mencari slot kosong lain dalam *array* dengan pola tertentu. Karena keunggulan dalam hal kecepatan dan efisiensi, *hash table* banyak dimanfaatkan dalam berbagai bidang seperti database, sistem

pencarian, penyimpanan data, serta implementasi struktur data lain seperti *dictionary* atau map dalam pemrograman.

2.2 Pengertian *Hash Function*

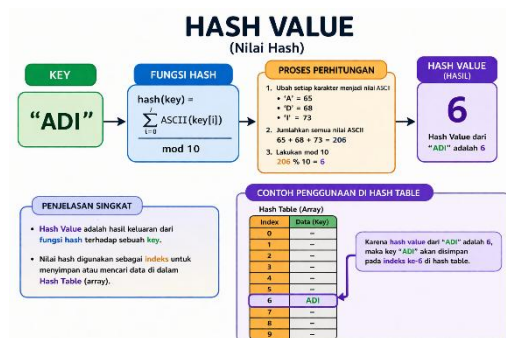


Gambar 2. 2 Hash Function

Hash function merupakan fungsi yang digunakan untuk mengubah suatu data atau kunci (*key*) menjadi nilai berupa angka yang dikenal sebagai *hash value*. Nilai ini kemudian dimanfaatkan sebagai indeks untuk menentukan lokasi penyimpanan data dalam struktur seperti *hash table*.

Secara sederhana, *hash function* berperan mengonversi data, baik berupa teks maupun angka, menjadi nilai numerik agar proses penyimpanan dan pencarian dapat dilakukan dengan lebih cepat. Fungsi ini dirancang agar bekerja secara efisien dan menghasilkan penyebaran nilai yang merata, sehingga dapat meminimalkan kemungkinan terjadinya *collision*. Namun, dalam praktiknya masih ada kemungkinan dua data berbeda menghasilkan nilai hash yang sama, sehingga diperlukan metode tambahan seperti *probing* atau *chaining* untuk menanganinya.

2.3 Pengertian *Hash Value*

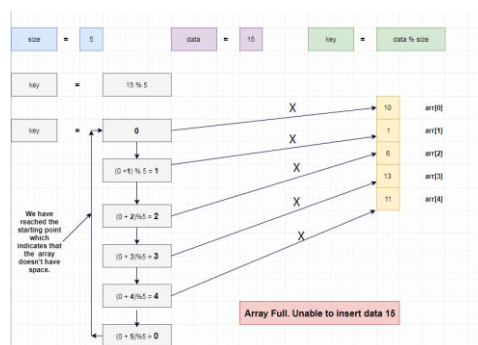


Gambar 2. 3 Hash Value

Hash value adalah nilai berupa angka atau kode tertentu yang dihasilkan dari proses *hashing*, yaitu ketika sebuah data atau kunci (*key*) diolah menggunakan fungsi *hash* untuk menghasilkan suatu nilai indeks. Nilai ini digunakan sebagai penentu posisi penyimpanan data dalam struktur seperti *hash table*, sehingga data dapat diakses dengan lebih cepat.

Secara sederhana, *hash value* dapat dianggap sebagai hasil perhitungan yang mewakili suatu data, sehingga komputer tidak perlu mencari data secara satu per satu. Meskipun demikian, dalam beberapa kasus, dua data yang berbeda bisa menghasilkan *hash value* yang sama, yang disebut sebagai *collision*, sehingga diperlukan metode penanganan khusus.

2.4 Pengertian *Linear Probing*

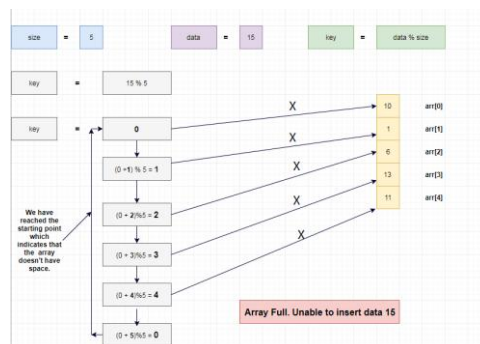


Gambar 2. 4 Linear Probing

Linear probing merupakan salah satu teknik dalam *hash table* yang digunakan untuk menangani *collision*, yaitu ketika dua data memiliki indeks yang sama. Metode ini bekerja dengan cara mencari slot kosong berikutnya secara berurutan di dalam *array* sampai ditemukan posisi yang dapat digunakan untuk menyimpan data.

Secara umum, *linear probing* menggunakan rumus $(hash + i) \bmod$ ukuran *array*, di mana i menunjukkan jumlah percobaan. Jika suatu indeks sudah terisi, maka proses pencarian akan dilanjutkan ke indeks berikutnya secara bertahap. Meskipun sederhana dan mudah diterapkan, metode ini memiliki kelemahan berupa kemungkinan terjadinya penumpukan data pada area tertentu (*clustering*), yang dapat menurunkan kinerja *hash table* jika *collision* terjadi terlalu sering.

2.5 Pengertian *Quadratic Probing*

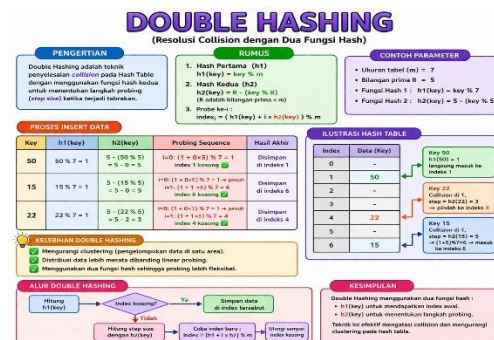


Gambar 2. 5 *Quadratic Probing*

Quadratic probing merupakan salah satu teknik dalam *hash table* yang digunakan untuk menangani *collision*, yaitu ketika dua data memiliki indeks yang sama. Metode ini bekerja dengan mencari posisi kosong lain di dalam *array* menggunakan pola perhitungan berbasis kuadrat, sehingga data baru tetap bisa disimpan tanpa mengganggu data yang sudah ada.

Secara umum, *quadratic probing* menggunakan rumus $(hash + i^2) \bmod$ ukuran *array*, di mana i menunjukkan jumlah percobaan. Dengan pendekatan ini, pencarian tidak dilakukan secara berurutan, melainkan melompat dengan jarak yang semakin besar. Cara ini dapat mengurangi penumpukan data pada satu area tertentu, sehingga performa hash table menjadi lebih baik dibandingkan metode probing sederhana.

2.6 Pengertian *Double Hashing*

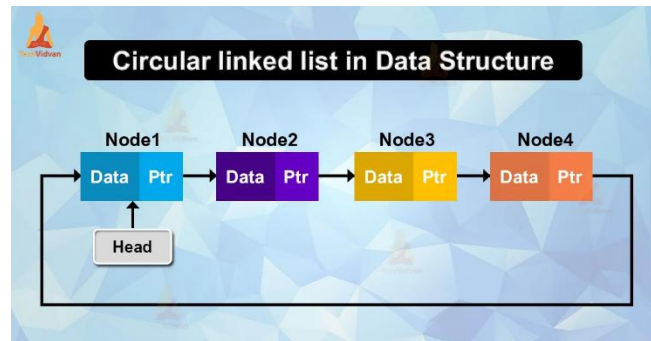


Gambar 2. 6 Double Hashing

Double hashing merupakan salah satu teknik dalam *hash table* yang digunakan untuk menangani *collision*, yaitu saat dua data memiliki indeks yang sama. Metode ini memanfaatkan dua fungsi *hash* untuk menentukan posisi penyimpanan data agar lebih efektif.

Pada metode ini, fungsi *hash* pertama digunakan untuk menentukan posisi awal data, sedangkan fungsi *hash* kedua digunakan untuk menentukan langkah perpindahan jika terjadi tabrakan. Dengan cara ini, pencarian posisi baru tidak dilakukan secara berurutan, melainkan berdasarkan perhitungan tertentu sehingga penyebaran data menjadi lebih acak dan merata. Hal ini membantu mengurangi penumpukan data pada satu area serta meningkatkan kinerja *hash table* secara keseluruhan.

2.7 Pengertian *Linked List*



Gambar 2. 7 Linked List

Linked list merupakan struktur data yang terdiri dari sekumpulan elemen yang disebut *node*, di mana setiap *node* saling terhubung menggunakan pointer atau referensi. Setiap *node* umumnya memiliki dua bagian, yaitu data sebagai nilai yang disimpan dan pointer yang menunjuk ke *node* berikutnya dalam rangkaian.

Berbeda dengan *array*, *linked list* tidak disimpan secara berurutan di dalam memori sehingga memiliki ukuran yang fleksibel dan dapat berubah sesuai kebutuhan. Struktur ini memudahkan proses penambahan maupun penghapusan data tanpa perlu menggeser elemen lainnya. Namun, untuk menemukan suatu data, proses pencarian harus dilakukan secara berurutan dari awal hingga akhir, sehingga cenderung lebih lambat dibandingkan dengan *array*.

2.8 Penjelasan Mentahan Program

2.8.1 Metode *Linked List Chaining*

2.8.1.1 Konsep Dasar *Linked List Chaining* :

Setiap *index* bisa menyimpan lebih dari satu data dalam bentuk *linked list*. Jadi saat mencari, kita:

- Tidak hanya cek 1 data
- Tapi harus menelusuri (*traverse*) *linked list*

2.8.1.2 Rumus yang Digunakan

$H(key)$ = jumlah *ASCII* semua huruf % ukuran *array*. Cara mencari nilai *ASCII* bisa dengan codingan dibawah ini :

```
#include <iostream>
using namespace std;

int main() {
    char huruf = 'A';
    cout << int(huruf);
}
```

Gambar 2. 8 Metode Linked List

Hasil *output* tergantung dengan huruf yang kita inputkan, semisalnya “A” kapital maka *output* nya :

65

Sedangkan kalau kita inputkan “a” kecil, maka *output* nya :

97

Dalam program diatas cuman bisa mencari satu persatu nilai *ASCII* dari huruf yang diinputkan, jadi semisal kita ingin mencari nama “Ando” maka kita harus cek satu persatu hurufnya (menginput satu-satu hurufnya) dari A, kemudian n, d, dan o, setelah semuanya dicek baru kita gabungkan nilai *ASCII* nya.

2.8.1.3 Langkah-langkah Mencari Data

No. _____
Date: _____

Jumlahkan nilai ASCII semua huruf, kemudian modulo dengan ukuran array.
Hasilnya nanti ialah index tujuan

1). Ando
↳ Hitung ASCII : $A(65) + n(110) + d(100) + o(111) = 386$
 $= 386 \text{ mod } 10 = 6$
Masuk ke index 6

2). Beni
↳ Hitung ASCII : $B(66) + e(101) + n(110) + i(105) = 382$
 $= 382 \text{ mod } 10 = 2$
Masuk ke index 2

3). Citra
↳ Hitung ASCII : $C(67) + i(105) + t(116) + r(114) + a(97) = 499$
 $= 499 \text{ mod } 10 = 9$
Masuk ke index 9

4). Dani
↳ Hitung ASCII : $D(68) + a(97) + n(110) + i(105) = 394$
 $= 394 \text{ mod } 10 = 4$
Masuk ke index 4

5). Eko
↳ Hitung ASCII : $E(69) + k(107) + o(111) = 287$
 $= 287 \text{ mod } 10 = 7$
Masuk ke index 7

6). Eko
↳ Hitung ASCII : $E(69) + k(107) + o(111) = 287$
 $= 287 \text{ mod } 10 = 7$
Masuk ke index 7

7). Gina
↳ Hitung ASCII : $G(71) + i(105) + n(110) + a(97) = 383$
 $= 383 \text{ mod } 10 = 3$
Disini index 3 sudah ada Eko, maka solusinya tambahkan dibelakang
Eko, jadinyanya seperti ini: Eko → Gina

No. _____
Date: _____

8). Intan
↳ Hitung ASCII : $I(73) + n(110) + t(116) + a(97) + n(110) = 506$
 $= 506 \text{ mod } 10 = 6$
Disini juga sama index 6 sudah ada Ando, maka solusinya tambahkan dibelakang Ando, jadinyanya seperti ini: 6: Ando → Intan

9). Joko
↳ Hitung ASCII : $J(74) + o(111) + k(107) + o(111) = 403$
 $= 403 \text{ mod } 10 = 3$
Index 3 sudah ada Eko → Gina, maka kita tambahkan lagi
dibelakang, maka jadinyanya: 3: Eko → Gina → Joko

10). Kevin
↳ Hitung ASCII : $K(75) + e(101) + v(118) + i(105) + n(110) = 509$
 $= 509 \text{ mod } 10 = 9$
Index 9 sudah ada Citra, maka solusinya tambahkan dibelakang Citra, maka jadinyanya: 9: Citra → Kevin

11). Hasil akhirnya

0	:	-
1	:	-
2	:	Beni
3	:	Eko → Gina → Joko
4	:	Dani
5	:	-
6	:	Ando → Intan
7	:	Eko
8	:	-
9	:	Citra → Kevin

1. Hitung Hash Data

- Jumlahkan nilai *ASCII* semua huruf
- Lalu *modulo* dengan ukuran *array*

Hasilnya = *index* tujuan

2. Akses *Index*

- Masuk ke *array* pada *index* tersebut

Kemungkinan:

- Kosong = data tidak ada
- Ada isi = lanjut ke langkah berikutnya

3. Telusuri *Linked List*

- Mulai dari node pertama (*head*)
- Bandingkan satu per satu:

Jika:

- Data sama = ditemukan
- Tidak sama = lanjut ke *node* berikutnya

4. Kondisi Berhenti

Pencarian berhenti jika:

- Data ditemukan
- Sudah sampai node terakhir (*NULL*)

Jika sampai akhir tidak ketemu = data tidak ada

2.8.2 Metode *Quadratic*

2.8.2.1 Konsep Dasar *Quadratic*

Quadratic Probing adalah metode pencarian dalam *hash table* dengan cara:

Jika *index* awal tidak cocok, maka mencari *index* lain menggunakan rumus kuadrat.

2.8.2.2 Rumus yang Digunakan

- *Hash Function*:

$H(key) = \text{jumlah ASCII} \% \text{ukuran array}$

- *Quadratic Probing*:

$H(i) = (h(key) + i^2) \% \text{ukuran array}$

Dimana:

- $i = 0, 1, 2, 3, \dots$
- $h(key) = \text{hash awal}$

2.8.2.3 Langkah-langkah Mencari Data

No. _____
Date: _____

Metode Quadratic

Metadanya ini awalnya mirip dengan Linked List Chaining, yang dimana kita perlu mencari semua nilai ASCII semua huruf lalu kita jumlahkan, dan kemudian modulo dengan ukuran array, nanti hasilnya itu adalah index yang bakal dituju, namun disini sedikit berbeda jika terjadi collision (tabrakan) maka kita harus mencari tempat kosong, istilahnya tidak boleh ada 2 data dalam 1 index

1) Ando
↳ Hitung ASCII
 $A(65) + n(100) + d(100) + o(111) = 386$
 $= 386 \bmod 10 = 6$
↳ Index 6 kosong → masuk

2) Beni
↳ Hitung ASCII
 $B(66) + e(101) + n(110) + i(105) = 382$
 $= 382 \bmod 10 = 2$
↳ Index 2 kosong → masuk

3) Citra
↳ Hitung ASCII
 $C(67) + i(105) + t(116) + r(114) + a(97) = 499$
 $= 499 \bmod 10 = 9$
↳ Index 9 kosong → masuk

4) Dini
↳ Hitung ASCII
 $D(68) + o(111) + n(110) + i(105) = 394$
 $= 394 \bmod 10 = 4$
↳ Index 4 kosong → masuk

5) Eka
↳ Hitung ASCII
 $E(69) + k(107) + a(97) = 273$

No. _____
Date: _____

↳ Index 3 kosong → masuk

6) Eka
↳ Hitung ASCII
 $E(69) + k(107) + a(97) = 273$
 $= 273 \bmod 10 = 3$
↳ Index 3 kosong → masuk

7) Gino
↳ Hitung ASCII
 $G(71) + i(105) + n(110) + o(111) = 387$
 $= 387 \bmod 10 = 7$
↳ Index 7 sudah ada Eka → collision

Quadratic:

- $i = 1 \rightarrow (387 + 1^2) \% 10 = 388 \% 10 = 8$ (Sudah ada Dini)
- $i = 2 \rightarrow (387 + 2^2) \% 10 = 391 \% 10 = 1$ (Sudah ada Beni)
- $i = 3 \rightarrow (387 + 3^2) \% 10 = 396 \% 10 = 6$ (Sudah ada Ando)
- $i = 4 \rightarrow (387 + 4^2) \% 10 = 403 \% 10 = 3$ (Sudah ada Eka)
- $i = 5 \rightarrow (387 + 5^2) \% 10 = 412 \% 10 = 2$ (Sudah ada Beni)

↳ Karena index 8 kosong, maka Gino masuk ke index 8.

8) Intan
↳ Hitung ASCII
 $I(73) + n(110) + t(116) + a(97) = 506$
 $= 506 \bmod 10 = 6$
↳ Index 6 sudah ada Ando → collision

Quadratic:

- $i = 1 \rightarrow (506 + 1^2) \% 10 = 507 \% 10 = 7$ (Sudah ada Eka)
- $i = 2 \rightarrow (506 + 2^2) \% 10 = 510 \% 10 = 0$ (Kosong)

↳ Karena index 0 kosong, maka Intan masuk ke index 0.

No. _____
Date: _____

9) Joko
↳ Hitung ASCII
 $J(74) + o(111) + k(107) + o(111) = 403$
 $= 403 \bmod 10 = 3$
↳ Index 3 sudah ada Eka → collision

Quadratic:

- $i = 1 \rightarrow (403 + 1^2) \% 10 = 404 \% 10 = 4$ (Sudah ada Dini)
- $i = 2 \rightarrow (403 + 2^2) \% 10 = 407 \% 10 = 7$ (Sudah ada Eka)
- $i = 3 \rightarrow (403 + 3^2) \% 10 = 412 \% 10 = 2$ (Sudah ada Beni)
- $i = 4 \rightarrow (403 + 4^2) \% 10 = 419 \% 10 = 9$ (Sudah ada Citra)
- $i = 5 \rightarrow (403 + 5^2) \% 10 = 428 \% 10 = 8$ (Sudah ada Gino)
- $i = 6 \rightarrow (403 + 6^2) \% 10 = 439 \% 10 = 9$ (Sudah ada Citra)
- $i = 7 \rightarrow (403 + 7^2) \% 10 = 452 \% 10 = 2$ (Sudah ada Beni)
- $i = 8 \rightarrow (403 + 8^2) \% 10 = 467 \% 10 = 7$ (Sudah ada Eka)
- $i = 9 \rightarrow (403 + 9^2) \% 10 = 484 \% 10 = 4$ (Sudah ada Dini)

↳ Hasilnya semua penuh (Quadratic gagal cari slot)

10) Kevin
↳ Hitung ASCII
 $K(75) + e(101) + v(118) + i(105) + n(110) = 509$
 $= 509 \bmod 10 = 9$
↳ Index 9 sudah ada Citra → collision

Quadratic:

- $i = 1 \rightarrow (509 + 1^2) \% 10 = 510 \% 10 = 0$ (Sudah ada Intan)
- $i = 2 \rightarrow (509 + 2^2) \% 10 = 513 \% 10 = 3$ (Sudah ada Eka)
- $i = 3 \rightarrow (509 + 3^2) \% 10 = 518 \% 10 = 8$ (Sudah ada Gino)
- $i = 4 \rightarrow (509 + 4^2) \% 10 = 525 \% 10 = 5$ (Kosong)

↳ Karena index 5 kosong, maka Kevin masuk ke index 5.

No. _____
Date: _____

Hasil akhirnya

- 0 : Intan
- 1 : -
- 2 : Beni
- 3 : Eka
- 4 : Dini
- 5 : Kevin
- 6 : Ando
- 7 : Eka
- 8 : Gino
- 9 : Citra

↳ Jika tidak bisa masuk karena semua slot yang dilalui quadratic sudah penuh.

1. Hitung Hash Awal

- Gunakan rumus hash
- Dapatkan index awal

2. Cek *Index* Tersebut

- Jika data cocok = selesai (menemukan)
- Jika tidak cocok = lanjut *probing*

3. Lakukan *Quadratic Probing*

Gunakan rumus:

- $i = 1 \rightarrow (h + 1^2) \% 10$
- $i = 2 \rightarrow (h + 2^2) \% 10$
- $i = 3 \rightarrow (h + 3^2) \% 10$
- dst

4. Bandingkan Tiap *Index*

- Jika cocok = data ditemukan
- Jika kosong = data tidak ada
- Jika tidak cocok = lanjut i berikutnya

5. Berhenti Jika:

- Data ditemukan
- Ketemu slot kosong
- Sudah mencoba semua kemungkinan

2.8.3 Kesimpulan dari *Metode Quadratic & Linked List Chaining*

Berdasarkan hasil percobaan kami, metode *Linked List Chaining* lebih efisien dibandingkan dengan metode *Quadratic Probing*. Hal ini dikarenakan seluruh data dapat dimasukkan ke dalam tabel tanpa kehilangan data, meskipun terjadi *collision*. Sedangkan pada metode *Quadratic Probing*, terdapat data yang tidak dapat dimasukkan karena keterbatasan slot dalam *array*. Selain itu, *chaining* lebih fleksibel karena setiap *index* dapat menampung lebih dari satu data. Oleh karena itu, *Linked List Chaining* lebih cocok digunakan terutama ketika jumlah data cukup banyak dan kemungkinan *collision* tinggi.

2.9 Penjelasan *Program Coding*

1. *Header*

```
#include <iostream>
#include <iomanip>
using namespace std;
```

Kegunaannya :

- *iostream* = untuk menampilkan *output* ke layer
- *iomanip* = untuk merapikan tampilan *output*
- *using namespace std* = supaya penulisan lebih singkat

2. *Struct Node*

```
struct Node {
    string nama;
    Node* next;
};
```

Kegunaannya :

Untuk membuat elemen dalam *linked list*

- *nama* = menyimpan data
- *next* = menunjuk ke data berikutnya

Tanpa ini:

kita tidak bisa menyimpan lebih dari 1 data dalam 1 *index*

3. *Hash Table*

```
const int SIZE = 10;
Node* table[SIZE];
```

Kegunaannya :

- *SIZE* = menentukan jumlah slot/*index*

- *table* = tempat utama menyimpan data

Kenapa kita menggunakan *Node**

karena setiap *index* bisa menyimpan linked list (bukan 1 data saja)

4. Hash Function

```
int hashFunction(string nama)
```

Kegunaannya :

- Mengubah data (nama) menjadi *index array*

```
int sum = 0;
```

- Menyimpan total *ASCII*

```
for (char c : nama)
```

- Mengambil setiap huruf

```
sum += int(c);
```

- Mengubah huruf menjadi angka *ASCII*

```
return sum % SIZE;
```

- Membatasi *index* agar 0–9 saja

Tanpa *modulo*:

- *index* bisa terlalu besar dan bisa terjadi *error*

5. Insert Function

```
void insert(string nama)
```

Kegunaannya untuk memasukkan data ke *hash table*

- Ambil *index*

```
int index = hashFunction(nama);
```

Kegunaannya untuk menentukan posisi data

- Buat *node* baru

```
Node* newNode = new Node;
```

Kegunaannya untuk membuat tempat untuk menyimpan data baru

```
newNode->nama = nama;
```

Kegunaannya ialah menyimpan isi data

```
newNode->next = NULL;
```

Kegunaannya untuk menandakan bahwa *node* ini belum terhubung

➤ Cek apakah *index* kosong

```
if (table[index] == NULL)
```

Kegunaannya untuk mengetahui apakah terjadi *collision* atau tidak

➤ Kalau kosong

```
table[index] = newNode;
```

Kegunaannya ialah langsung simpan tanpa masalah

➤ Kalau *collision*

```
Node* temp = table[index];
```

Kegunaannya untuk menyimpan posisi awal *linked list*

```
while (temp->next != NULL)
```

Kegunaannya untuk mencari *node* terakhir

```
temp = temp->next;
```

Kegunaannya untuk berpindah ke *node* berikutnya

```
temp->next = newNode;
```

Kegunaannya untuk menambahkan data di akhir *list*

6. *Display Function*

```
void display()
```

Kegunaannya untuk menampilkan isi *hash table* ke *layer*

```
for (int i = 0; i < SIZE; i++)
```

Kegunaannya untuk menelusuri semua *index*

```
cout << setw(2) << i << " : ";
```

Kegunaannya untuk menampilkan nomor *index* dengan rapi

```
Node* temp = table[i];
```

Kegunaannya untuk mengambil isi *index*

```
if (temp == NULL)
```

Kegunaannya untuk mengecek apakah kosong

```
while (temp != NULL)
```

Kegunaannya untuk menelusuri seluruh *linked list*

```
cout << temp->nama;
```

Kegunaannya untuk menampilkan data

```
temp = temp->next;
```

Kegunaannya untuk pindah ke *node* berikutnya

7. Main Function

```
int main()
```

Tempat *program* dijalankan

➤ Inisialisasi

```
table[i] = NULL;
```

Kegunaannya untuk mengosongkan semua *index*, kalau tidak, maka bisa berisi data sampah (*error*)

➤ Data

```
string data[]
```

Kegunaannya untuk menyimpan daftar nama

➤ Hitung jumlah data

```
sizeof(data) / sizeof(data[0])
```

Kegunaannya untuk mengetahui jumlah elemen otomatis

➤ Insert

```
insert(data[i]);
```

Kegunaannya untuk memasukkan semua data ke hash table

➤ Display

```
display();
```

Menampilkan hasil akhir

2.10 Hasil Output

```
===== HASIL HASH TABLE (Linked List Chaining) =====  
  
0 : -  
1 : -  
2 : Beni  
3 : Eka -> Gina -> Joko  
4 : Doni  
5 : -  
6 : Ando -> Intan  
7 : Eko  
8 : -  
9 : Citra -> Kevin  
  
=====
```

```
PS C:\C++>
```

Gambar 2. 9 Output

Berdasarkan *output hash table* dengan metode *Linked List Chaining*, dapat disimpulkan bahwa data berhasil didistribusikan ke dalam *array* berdasarkan hasil *hash function* (jumlah *ASCII* % 10). Beberapa data memiliki index yang sama sehingga terjadi *collision*, namun *collision* tersebut dapat diatasi dengan baik menggunakan *linked list* (ditunjukkan dengan tanda panah (->). Dengan metode ini, seluruh data dapat disimpan tanpa ada yang hilang, meskipun terdapat beberapa *index* yang kosong dan beberapa *index* yang berisi lebih dari satu data.

BAB III

PENUTUP

3.1 Kesimpulan

Berdasarkan pembahasan yang telah dilakukan, dapat disimpulkan bahwa *hash table* merupakan struktur data yang efisien dalam proses penyimpanan dan pencarian data karena memanfaatkan fungsi *hash* untuk menentukan posisi *indeks* secara langsung. Dengan kompleksitas waktu yang mendekati konstan atau $O(1)$, *hash table* sangat sesuai digunakan untuk mengelola data dalam jumlah besar. Namun, dalam penerapannya masih dapat terjadi *collision*, sehingga diperlukan metode penanganan seperti *linear probing*, *quadratic probing*, *double hashing*, maupun *chaining* menggunakan *linked list*.

Dari hasil implementasi *program*, metode *Linked List Chaining* menunjukkan keunggulan karena mampu menampung lebih dari satu data pada satu *indeks* melalui struktur *linked list*, sehingga tidak ada data yang hilang saat terjadi *collision*. Sebaliknya, metode *Quadratic Probing* memiliki keterbatasan pada jumlah slot yang tersedia, sehingga memungkinkan adanya data yang tidak dapat disimpan apabila tabel sudah penuh atau tidak ditemukan posisi kosong.

Selain itu, penggunaan *hash function* yang memanfaatkan jumlah nilai *ASCII* dan dimodulo dengan ukuran *array* mampu membantu dalam proses distribusi data, meskipun belum sepenuhnya menghindari *collision*. Oleh karena itu, pemilihan fungsi *hash* serta metode penanganan *collision* yang tepat sangat berpengaruh terhadap kinerja dan efisiensi *hash table* secara keseluruhan.

3.2 Saran

Dalam penggunaan *hash table*, disarankan untuk memilih fungsi *hash* yang dapat mendistribusikan data secara merata agar dapat meminimalkan terjadinya *collision*. Selain itu, pemilihan metode penanganan *collision* seperti *linear probing*, *quadratic probing*, atau *double hashing* sebaiknya disesuaikan dengan kebutuhan serta karakteristik data yang digunakan.

Berdasarkan hasil percobaan, metode *Linked List Chaining* lebih direkomendasikan terutama untuk jumlah data yang besar, karena lebih fleksibel dan mampu menyimpan seluruh data tanpa kehilangan. Sementara itu, metode *Quadratic Probing* masih dapat digunakan jika jumlah data relatif sedikit dan kapasitas tabel masih mencukupi.

Untuk pengembangan selanjutnya, disarankan menambahkan fitur seperti pencarian dan penghapusan data, serta melakukan pengujian performa antar metode penanganan *collision*. Selain itu, penggunaan variasi fungsi *hash* lain juga dapat dipertimbangkan agar distribusi data menjadi lebih optimal dan sesuai dengan kebutuhan sistem.

DAFTAR PUSTAKA

GeeksforGeeks. (n.d.). “*Hashing data structure.*” Diakses dari <https://www.geeksforgeeks.org/hashing-data-structure/> Diakses pada 25 Maret 2026.

TutorialsPoint. (n.d.). “*Data structures - Hash table.*” dari https://www.tutorialspoint.com/data_structures_algorithms/hash_data_structure.htm

Programiz. (n.d.). “*Hash table.*” Diakses dari <https://www.programiz.com/dsa/hash-table> Diakses pada 26 Maret 2026.